Easy Is Taken

What’s left for platforms when software is a utility?

Luke Philips



CLOUD NATIVE
PLATFORM ENGINEERING COMMUNITY

“Easy is taken”

> “a plentiful thing ain’t worth so much.”

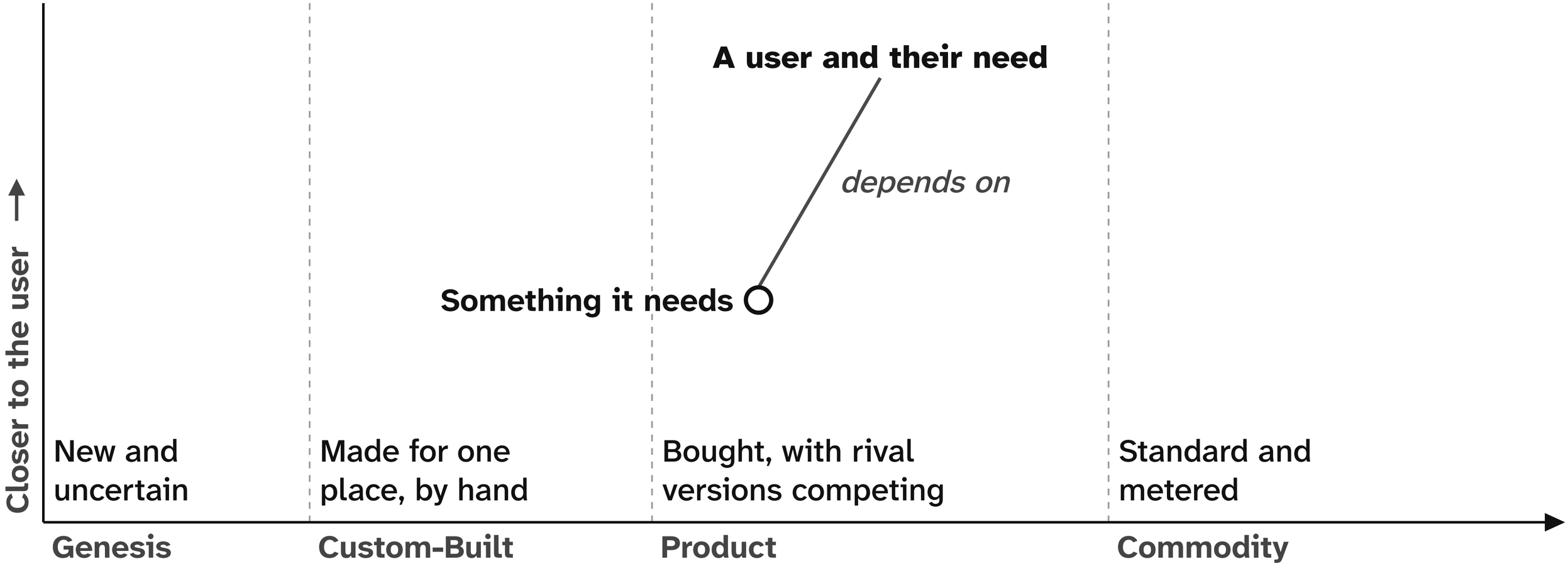
Steve Fenton, “Easy is taken”, stevefenton.co.uk, 2026-10-19

I read that as software becoming a utility.

A supposition for the next ten minutes.

How to read the maps

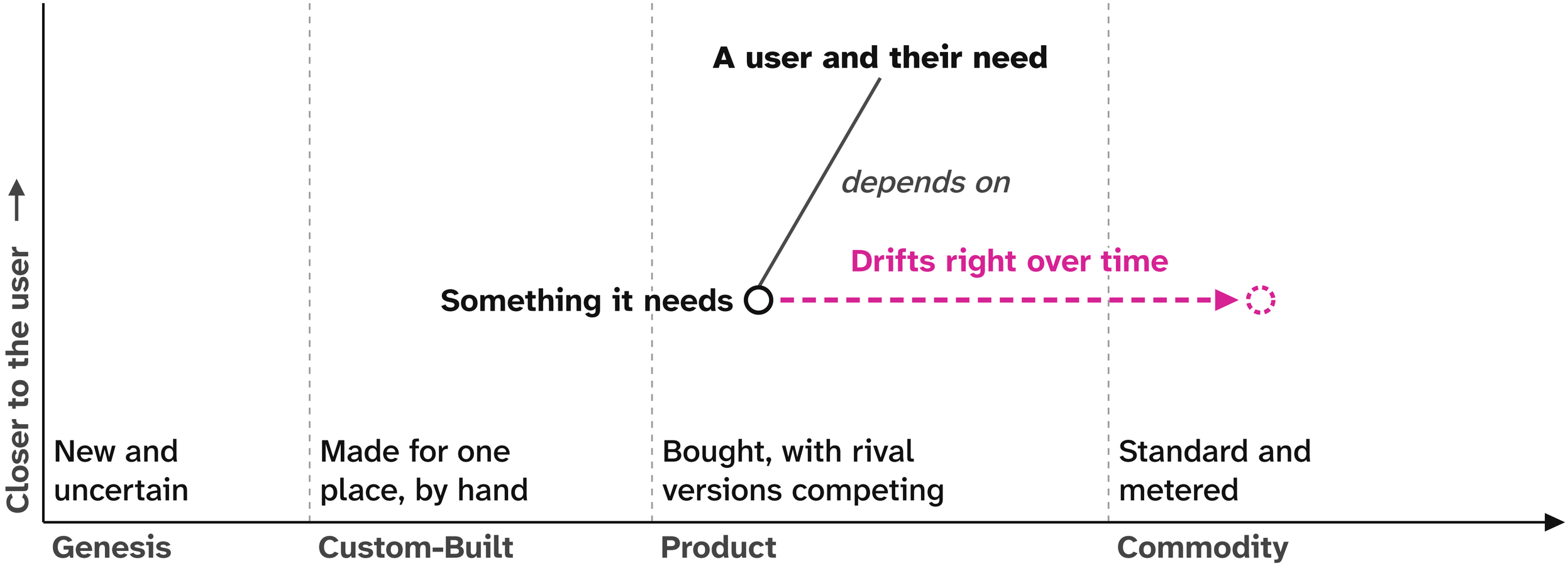
Two axes and a line



Every position on these maps is a judgment.

How to read the maps

Two axes and a line



Every position on these maps is a judgment.

Six landscapes, two of them dissents

user-need

Start from who the platform serves. Wardley calls the user need “the anchor for the entire map”.

Who is the user once a prompt meets the first need?

gameplay

The moves players choose against each other, such as a platform and its vendors.

What can a platform hold, and against whom?

climate today

What changes the map whatever anyone does.

What does the weather do to a platform?

captured-decisions

Tests one idea: a platform is worth the decisions it holds.

Is captured opinion the value?

dissent: faster craft

Disagrees with the supposition, in Simon Wardley’s phrase.

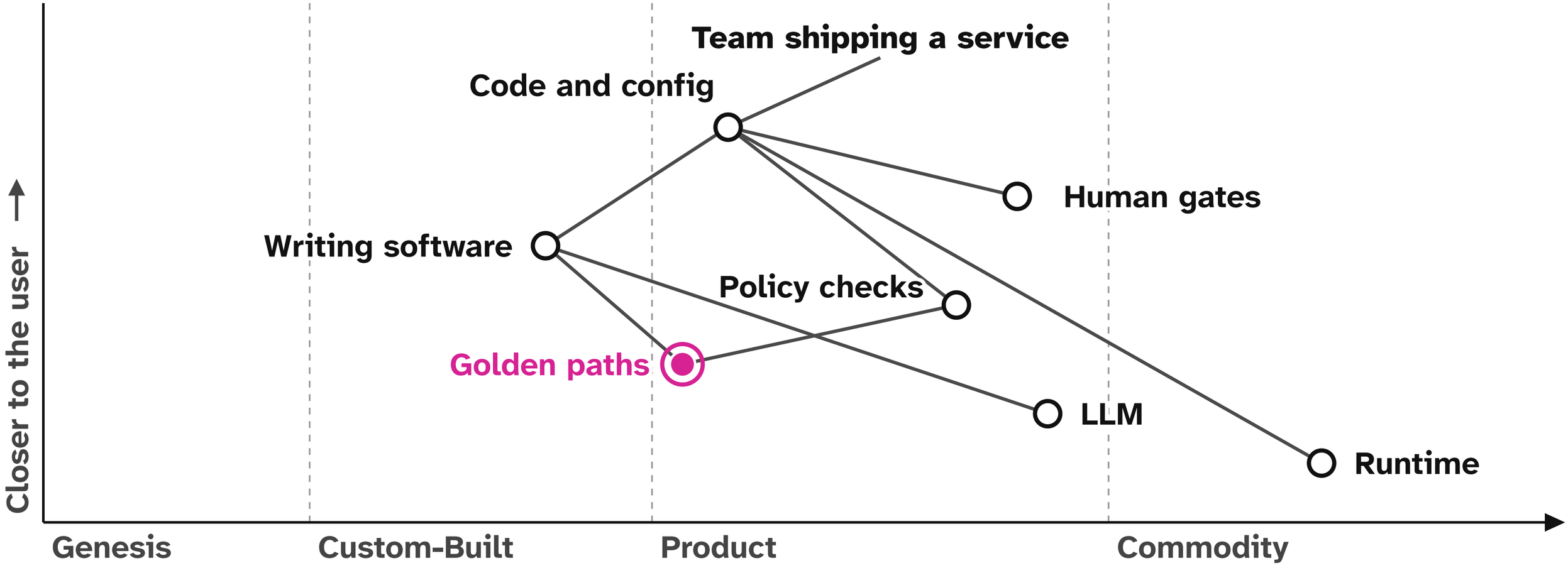
What if writing has only become “a faster craft”?

dissent: no platform

Pushes the supposition to its end.

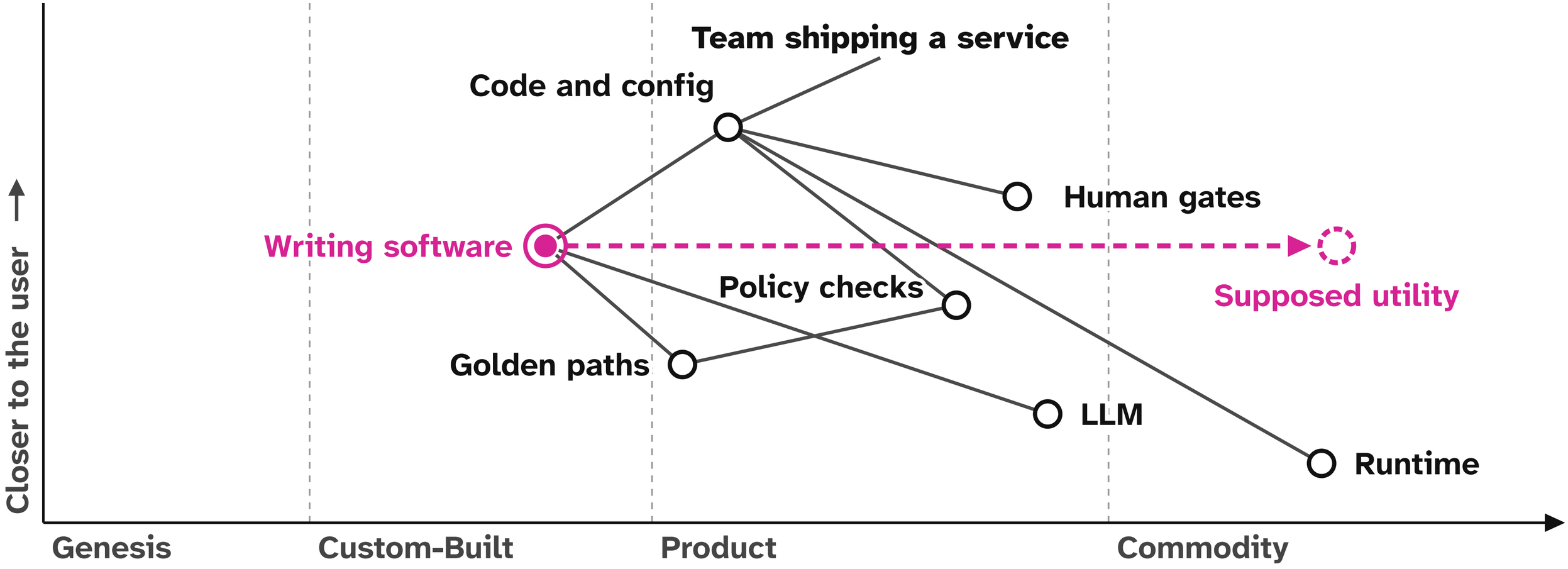
Does the platform leave the map?

A platform, before the supposition



A golden path is two things in one box: effort someone saved you, and decisions someone made for you. When a team picks yours, which one are they paying for?

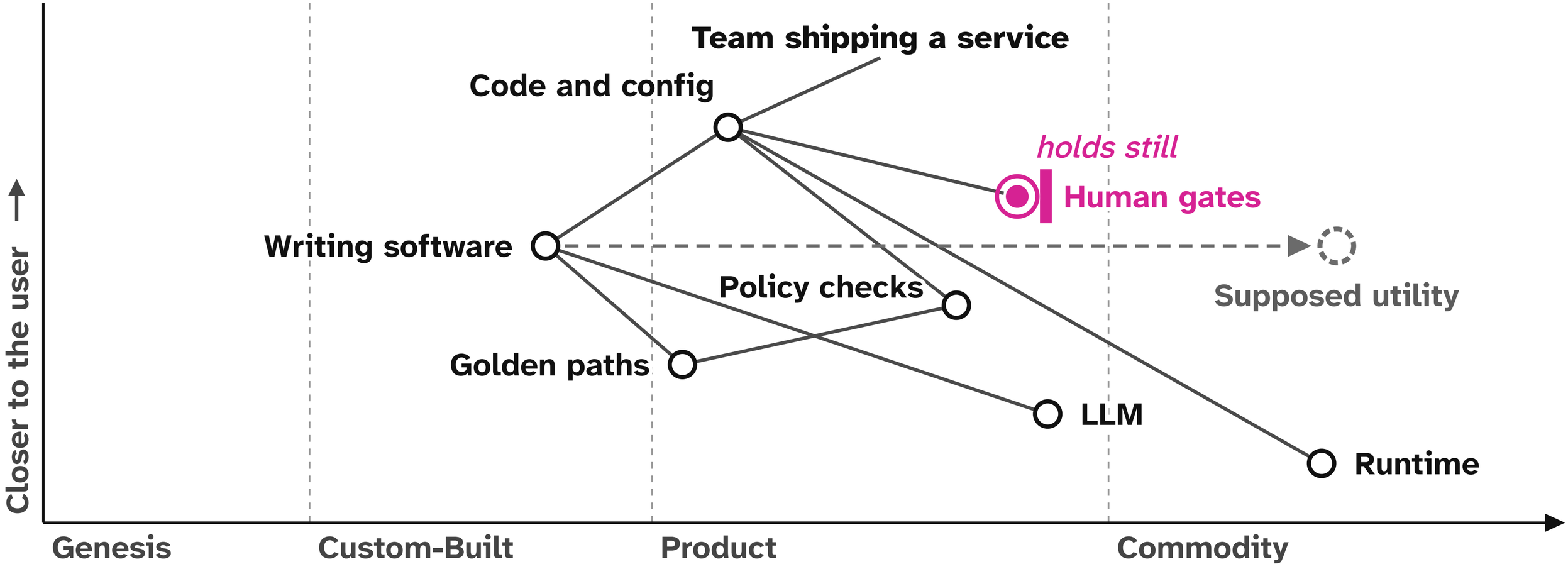
Suppose writing software moves



If writing code gets cheap, so does anything priced on the effort to write it, like templates and modules. How much of your platform was priced that way?

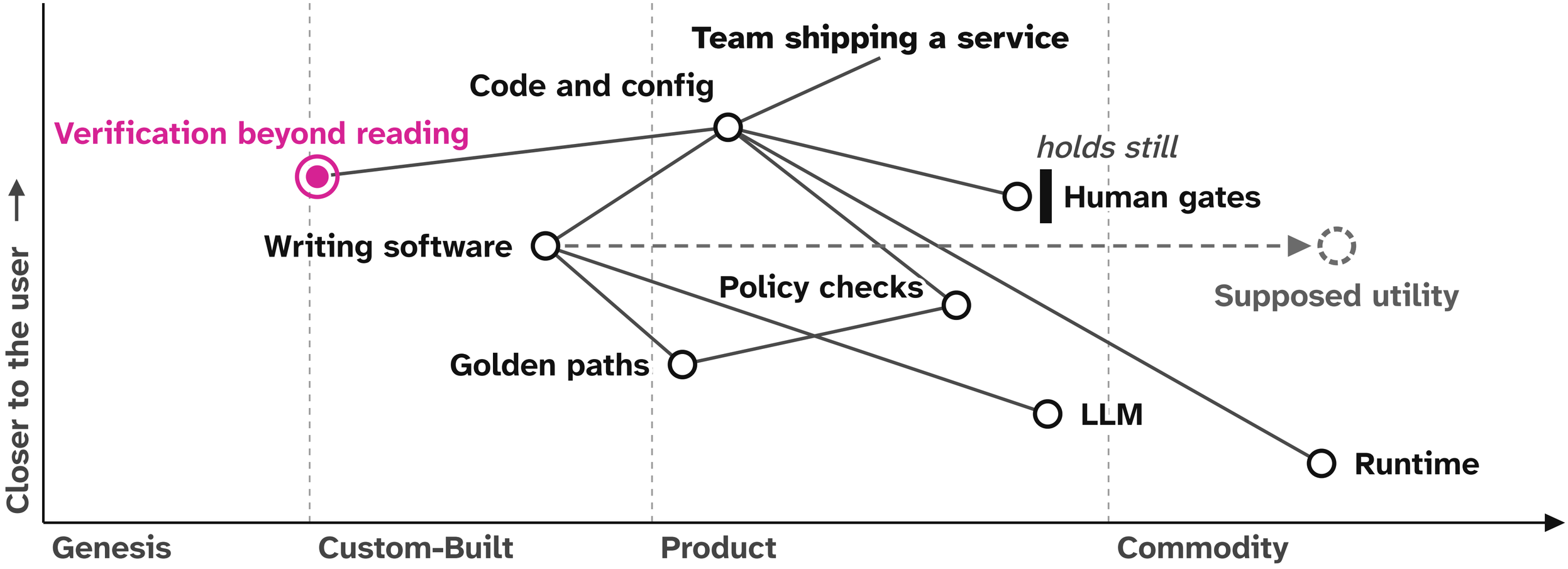
Where the extra weight lands

> “If you up the rate and size of changes passing through the review stage, pressure is simply transferred to the real bottleneck.” **Steve Fenton, The New Stack, 2026-07-16**



More code arrives at the same gates, and the gates hold still. What in your organization didn’t speed up, and what’s queuing in front of it now?

Adjust, or fall apart?



Checking a change without anyone reading every line is a practice we’re still inventing. Is your platform where it gets worked out, or what teams route around?

What’s left? Six answers that don’t agree (1 of 2)

user-need

Answering for what runs.

Our maps draw one user, the developer. Suppose a prompt meets half of what they need. The other half belongs to someone we never drew.

If your platform vanished tomorrow and your teams kept shipping, who would complain first?

What’s left? Six answers that don’t agree (1 of 2)

user-need

Answering for what runs.

Our maps draw one user, the developer. Suppose a prompt meets half of what they need. The other half belongs to someone we never drew.

If your platform vanished tomorrow and your teams kept shipping, who would complain first?

gameplay

Whatever your team still chooses to build.

Teams stayed on the platform partly because the way round cost more. Suppose a prompt makes it cheap. Now someone else is watching too.

Who learns first what your teams keep asking for: you, or whoever sells them the agent?

What’s left? Six answers that don’t agree (1 of 2)

user-need

Answering for what runs.

Our maps draw one user, the developer. Suppose a prompt meets half of what they need. The other half belongs to someone we never drew.

If your platform vanished tomorrow and your teams kept shipping, who would complain first?

gameplay

Whatever your team still chooses to build.

Teams stayed on the platform partly because the way round cost more. Suppose a prompt makes it cheap. Now someone else is watching too.

Who learns first what your teams keep asking for: you, or whoever sells them the agent?

captured-decisions

Deciding, and making decisions bind.

Much platform opinion may be generic enough that a model already knows it. What stays scarce is deciding, and agents already do some of it.

What did your agents decide last week that nobody in the company knows was decided?

What’s left? Six answers that don’t agree (1 of 2)

user-need

Answering for what runs.

Our maps draw one user, the developer. Suppose a prompt meets half of what they need. The other half belongs to someone we never drew.

If your platform vanished tomorrow and your teams kept shipping, who would complain first?

gameplay

Whatever your team still chooses to build.

Teams stayed on the platform partly because the way round cost more. Suppose a prompt makes it cheap. Now someone else is watching too.

Who learns first what your teams keep asking for: you, or whoever sells them the agent?

captured-decisions

Deciding, and making decisions bind.

Much platform opinion may be generic enough that a model already knows it. What stays scarce is deciding, and agents already do some of it.

What did your agents decide last week that nobody in the company knows was decided?

What’s left? Six answers that don’t agree (2 of 2)

dissent: faster craft

The craft, done faster.

The tools moved a long way. The work moved a little. A carpenter with a nail gun is still a carpenter, and nobody has industrialized the craft.

What are you building on as if it were already a utility?

What’s left? Six answers that don’t agree (2 of 2)

dissent: faster craft

The craft, done faster.

The tools moved a long way. The work moved a little. A carpenter with a nail gun is still a carpenter, and nobody has industrialized the craft.

What are you building on as if it were already a utility?

climate

The gap between fast tools and slow practice.

Cheap code loads everything that didn’t get cheaper. Practice takes years to catch up, and a platform may live in that gap, for a while.

If checking a change got as cheap as writing one, what would your platform still do?

What’s left? Six answers that don’t agree (2 of 2)

dissent: faster craft

The craft, done faster.

The tools moved a long way. The work moved a little. A carpenter with a nail gun is still a carpenter, and nobody has industrialized the craft.

What are you building on as if it were already a utility?

climate

The gap between fast tools and slow practice.

Cheap code loads everything that didn’t get cheaper. Practice takes years to catch up, and a platform may live in that gap, for a while.

If checking a change got as cheap as writing one, what would your platform still do?

dissent: no platform

The work between people.

Push the supposition all the way and the platform leaves the map for anything one team can do alone. What stays is the work between teams.

When the config is perfect and the outage happens anyway, whose name is on it?

What’s left? Six answers that don’t agree (2 of 2)

dissent: faster craft

The craft, done faster.

The tools moved a long way. The work moved a little. A carpenter with a nail gun is still a carpenter, and nobody has industrialized the craft.

What are you building on as if it were already a utility?

climate

The gap between fast tools and slow practice.

Cheap code loads everything that didn’t get cheaper. Practice takes years to catch up, and a platform may live in that gap, for a while.

If checking a change got as cheap as writing one, what would your platform still do?

dissent: no platform

The work between people.

Push the supposition all the way and the platform leaves the map for anything one team can do alone. What stays is the work between teams.

When the config is perfect and the outage happens anyway, whose name is on it?

The repo

Every scenario, map, review and source.

CLOUD NATIVE
PLATFORM ENGINEERING COMMUNITY



github.com/lukepatrick/easy-is-taken



linktr.ee/lukephilips

Luke Philips